Security Fundamentals

Contents

Reference

1. The security mindset

Ordinary engineering asks how a system behaves when used as intended. Security asks how it behaves when somebody is **deliberately trying to make it behave differently**, which is a different question and produces different answers.

The shift that matters:

Normal thinking	Security thinking
What does the user do?	What can somebody make the system do?
Does the form validate?	What happens when the request skips the form entirely?
The client checks this	The client is under the attacker's control
This value comes from our own system	Which part, and can it be influenced from outside?

Every input is attacker-controlled until we have established otherwise, and "input" is broader than a form field. Request bodies, query strings, headers, cookies, file names, file contents, values read from a database that were written by a user, and responses from other services all qualify. The single most common category of vulnerability comes from treating data as trustworthy because of where it arrived from rather than because it was checked.

The corollary is the one candidates most often miss in interviews: **client-side validation is a usability feature, not a control.** It gives fast feedback and reduces pointless round trips. It stops nobody, because the client is code running on somebody else's machine and the request can be made without it.

Two habits worth carrying into ordinary work:

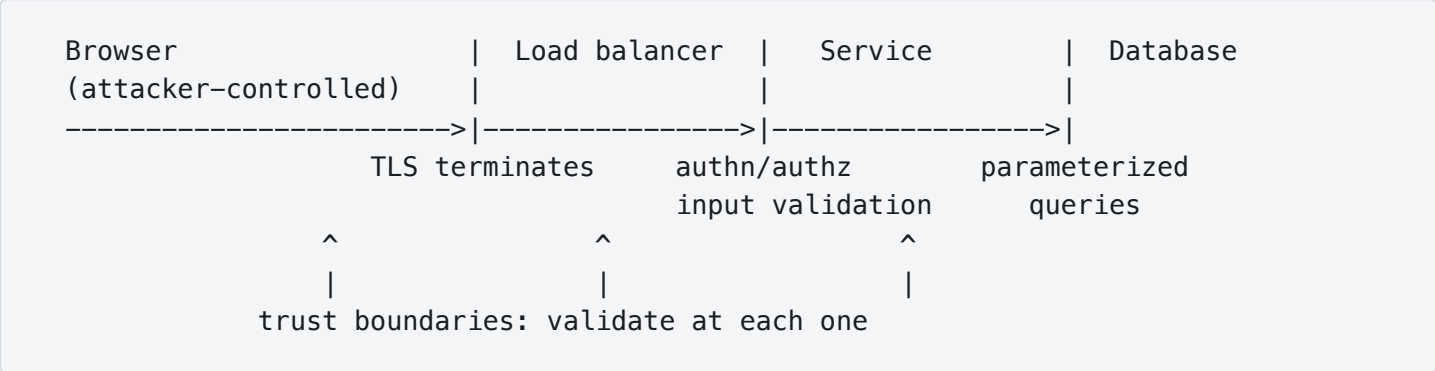
- **Ask what happens on the failure path.** Most security defects live in error handling, retries and edge cases, because those paths were written quickly and tested least.
- **Ask who can reach this.** A function is only as protected as the least protected route to it.

2. Threat modelling

Threat modelling is the structured version of "what could go wrong here", done before the code exists rather than after the incident. Four questions carry most of it:

1. **What are we building?** A diagram of components, data stores and the flows between them.
2. **What can go wrong?** The systematic part, below.
3. **What are we going to do about it?** Mitigate, accept, transfer, or remove the feature.
4. **Did we do a good job?** Reviewed as the design changes.

Trust boundaries are where the diagram becomes useful. A trust boundary is any line data crosses where the level of trust changes: browser to server, service to service, application to database, our code to a third-party library.



Every boundary crossing needs an explicit decision about what must be validated, authenticated, authorized or constrained there, and every box needs an answer to what it assumes about its inputs. The decision can legitimately be "nothing extra"; what causes trouble is never making it. Most vulnerabilities are a boundary somebody did not notice was a boundary.

STRIDE is the usual prompt list for question two, and it is worth memorising because it produces a systematic sweep rather than whatever we happened to think of:

	Threat	The property it breaks
S	Spoofing	Authentication
T	Tampering	Integrity
R	Repudiation	Non-repudiation
I	Information disclosure	Confidentiality
D	Denial of service	Availability
E	Elevation of privilege	Authorization

The three properties in the middle of that list, **confidentiality, integrity and availability**, are the classic triad, and STRIDE is essentially that triad expanded into things an attacker actually does.

Threat modelling does not need a formal process to be worth doing. Spending twenty minutes on a diagram and the STRIDE list, at design time, catches the class of problem that is expensive to fix once the shape of the system depends on it.

3. Principles that hold everywhere

These recur across every specific vulnerability in this document, which is why they are worth stating once.

Defence in depth. Assume any single control will fail, and arrange that the failure is not sufficient. Parameterized queries, plus least-privilege database credentials, plus not returning raw errors, plus alerting on anomalous query volume. Each is imperfect; the combination requires four failures rather than one.

Least privilege. Every component gets exactly the access it needs and no more. The application's database user does not need `DROP TABLE`. The service reading from a bucket does not need to write to it. This is what turns a compromise into a limited one.

Fail closed. When a check cannot complete, deny. The pattern to watch for is an authorization check wrapped in a `try` block whose `catch` logs and continues, which converts an error in the permission service into access granted.

```
// Fails open: an exception from the permission service grants access.
try {
    return permissions.canRead(user, document);
} catch (Exception e) {
    log.warn("permission check failed", e);
    return true;                       // wrong direction
}
```

Secure by default. The default configuration should be the safe one, so that being insecure requires a deliberate act. New buckets private, new endpoints authenticated, new fields not serialised, debug output off unless switched on.

Do not build our own cryptography. Use a reviewed library, and use it in the standard way. This applies to the constructions as much as the algorithms: rolling a custom token format, a custom password hash, or a custom signature scheme is where things go wrong, even when the underlying primitives are sound.

Validate on the server. Restated from section 1 because it is the principle most often violated in practice, and because it makes the client-side check honest about what it is: a convenience.

Minimise what we hold. Data not collected cannot be leaked. Data deleted on schedule cannot be leaked later. This is a security control as much as a privacy one, and it is the cheapest one available.

4. Authentication

Authentication is proving who somebody is. Authorization is deciding what they may do. Two different questions, in that order, and conflating them is a common source of both bugs and confused interview answers.

Authentication factors, and a system is multi-factor only when it combines **different** categories:

Factor	Examples
Something known	Password, passphrase, recovery answer
Something held	Phone with an authenticator application, hardware key, smart card
Something inherent	Fingerprint, face

Two passwords are not two factors. A password plus a code from an authenticator application is.

Multi-factor authentication is the single highest-value control for account security, because it breaks the attack that actually happens at scale: credentials reused from another site's breach. Strength varies by mechanism, though, and being able to rank them is worth marks:

Mechanism	Note
SMS codes	Better than nothing. Vulnerable to number porting and interception, and phishable
Time-based one-time passwords	Widely supported, and still phishable, since a person can be persuaded to read the code aloud
Hardware keys and passkeys	Bound to the site's origin, so a phishing page cannot use them. The strongest widely available option

That last row is the interesting one. WebAuthn credentials are tied to the origin they were registered against, so a convincing fake site cannot relay them, which removes phishing rather than merely discouraging it.

Around the login itself, several things need attention beyond the password check:

- **Rate limiting and lockout**, to make guessing expensive. Bear in mind that aggressive lockout is itself a denial-of-service route against a known account.
- **Uniform responses.** "No account with that email" tells an attacker which addresses are registered. The same message and the same timing for both cases avoids handing over that list.
- **Credential stuffing defence**, since most account takeover is reused passwords rather than guessing. Checking new passwords against known-breached lists is more effective than composition rules.
- **The recovery flow**, which is authentication by another route and is frequently the weakest one. An account protected by a hardware key and recoverable by a security question is protected by the security question.

5. Password storage

The question interviewers use to find out whether somebody has done this properly, because the wrong answers are specific and recognisable.

Passwords are hashed, never encrypted, because encryption implies a key that can reverse it and there is no legitimate reason to recover the original. And the hash must be from a family designed for passwords:

Use	Do not use
Argon2id, the current general recommendation	MD5, SHA-1
scrypt	SHA-256, SHA-512, or any general-purpose hash on its own
bcrypt	Anything hand-built from a general-purpose hash
PBKDF2, where the others are unavailable or a standard requires it	

The reason a general-purpose hash is wrong is precisely what makes it good elsewhere: it is **fast**. Hardware can compute billions of SHA-256 operations per second, so a leaked table of SHA-256 password hashes is a list of passwords shortly afterwards.

Password hashing functions are instead **deliberately expensive**, with a work factor that can be raised as hardware improves. They differ in how they impose that cost, and the distinction is worth getting right: **Argon2id and scrypt are memory-hard**, requiring a configurable amount of memory as well as processor time, which is what makes large-scale cracking on specialised hardware costly. **bcrypt is deliberately slow** without being memory-hard in the same sense, and **PBKDF2** raises cost through iteration count alone, which is why it is the weakest of the four against modern cracking hardware and is chosen mainly where a standard requires it.

A salt is a unique random value per password, stored alongside the hash, and it is not a secret. Its job is not to hide anything: it ensures two users with the same password get different hashes, which defeats precomputed tables and stops a single cracked hash revealing every account that shared that password. Modern libraries generate and store it inside the hash string for us.

```
// Spring Security. The salt is generated per password and encoded into the result.
PasswordEncoder encoder = new BCryptPasswordEncoder(12);   // 12 is the work factor

String stored = encoder.encode(rawPassword);
// $2a$12$Xy...   algorithm, work factor, salt and hash in one string

boolean ok = encoder.matches(rawPassword, stored);         // constant-time
comparison
```

A **pepper** is a secret value combined with the password, held outside the database, typically in a key manager or a configuration secret. It adds a useful property: a database leak alone is not enough to begin cracking. It is optional, it complicates key rotation, and it is not a substitute for any of the above.

Two details worth knowing because they come up as follow-up questions:

- **Use the library's verification method** rather than fetching the hash and comparing it ourselves. Established implementations handle the comparison semantics, including avoiding the timing differences an early-exit equality check would leak.

- **Work factors need revisiting.** A factor chosen five years ago is weaker now than it was. Specific parameter recommendations change with hardware, so the durable answer is to follow current OWASP guidance rather than to memorise numbers.

6. Sessions and tokens

Once somebody has authenticated, the next request has to be linked to that decision. Two broad approaches, and the trade between them is a standard interview question.

	Server-side sessions	Self-contained tokens
Where state lives	On the server, keyed by an opaque identifier	Inside the token the client holds
Revocation	Immediate. Delete the record	Hard. The token stays valid until it expires
Scaling	Needs shared storage across instances	No lookup needed
Size	A small identifier	Larger, sent on every request
Leak impact	The identifier is useless once revoked	Usable until expiry, wherever it is presented

The deciding factor is usually revocation. A session can be ended the instant an account is compromised or a role changes. A self-contained token cannot, without adding the very lookup it was designed to avoid.

Whichever is used, if it travels in a cookie the attributes matter and they are frequently asked:

```
Set-Cookie: session=abc123; HttpOnly; Secure; SameSite=Lax; Path=/; Max-Age=3600
```

Attribute	Effect
`HttpOnly`	JavaScript cannot read it, which limits what a successful XSS can steal
`Secure`	Sent over HTTPS only
`SameSite`	Controls whether it rides along on cross-site requests. Central to section 13
`Path`, `Domain`	Scope. Keep as narrow as the application allows
`Max-Age`, `Expires`	Lifetime. Shorter limits the window if it leaks

Two behaviours around the session lifecycle:

- **Rotate the identifier on privilege change**, particularly at login. Otherwise an attacker who plants a known identifier before login holds a valid authenticated session afterwards, which is session fixation.
- **Invalidate on the server at logout.** Clearing the cookie in the browser does nothing to a copy the attacker already has.

7. JSON Web Tokens

A JWT is three base64url-encoded parts joined by dots:

```
header.payload.signature

{"alg":"RS256","typ":"JWT"} . {"sub":"u-1","exp":1771200000,"role":"admin"} .
<signature>
```

> **A signed JWT is not encrypted**
>
> Base64url is **encoding, not encryption**. Anyone holding the token can read every claim in it by decoding the payload, with no key involved. The signature proves the token was issued by the holder of the signing key and has not been altered; it hides nothing.
>
> So **nothing confidential belongs in a normal signed JWT payload.** Identity claims such as `sub`, `iss`, `aud` and `exp` are exactly what it is for and are not secrets. Encrypted tokens exist as a separate construction (JWE), and the default thing people mean by "JWT" is signed and readable.

What the signature buys is integrity and origin: the server can verify the token without storing anything, which is the entire appeal. **What it costs** is revocation, from section 6. A token valid for an hour remains valid for an hour after the account is disabled, unless a list of revoked tokens is checked, which reintroduces the lookup.

The usual arrangement accepts that trade rather than pretending it away: a **short-lived access token**, minutes rather than hours, plus a **longer-lived refresh token** whose lifecycle the authorization server tracks, so it can be revoked or rotated. The client may hold the refresh token in secure storage; what matters is that the server keeps enough state to invalidate it. The exposure window then shrinks to the access token's lifetime.

The failures worth being able to name, because they are asked directly:

Failure	What happens
`alg: none`	The token declares it is unsigned and a naive library accepts it. Never let the token choose whether to be verified
Algorithm confusion	A token signed `HS256` is presented to a server expecting `RS256`, using the public key as the HMAC secret. The public key is public, so anybody can forge one. Pin the expected algorithm on the server
Not verifying at all	Decoding the payload and trusting it. Decoding is not verifying, and the two library calls often look similar
Ignoring `exp`, `iss` and `aud`	An expired token, or one issued for a different service, is accepted
Storing it in `localStorage`	Readable by any JavaScript on the page, so an XSS becomes a token theft. An `HttpOnly` cookie is not readable that way

Failure	What happens
Putting authorization data in it and never re-checking	A role revoked after issue is still asserted by the token

8. OAuth 2.0 and OpenID Connect

> **The distinction the question is testing**
>
> **OAuth 2.0 is an authorization framework, not an authentication protocol.** It answers "may this application act on this user's behalf, for this scope", and it deliberately says nothing about who the user is.
>
> **OpenID Connect is the identity layer built on top of it**, adding an **ID token** that states who authenticated, when, and how. "Sign in with ..." is OIDC.
>
> Using OAuth alone for login is a known anti-pattern, and the precise reason is worth stating: **an access token represents authorization to reach a resource. It is not an identity assertion about the user**, and treating the ability to call an API as proof of who is calling is what the pattern gets wrong.

The four roles, which the flows are easier to follow once named:

Role	Who
Resource owner	The user
Client	The application wanting access
Authorization server	Issues the tokens
Resource server	Holds the data, accepts the access token

Authorization Code with PKCE is the flow to know, and is the recommended interactive authorization flow. PKCE, Proof Key for Code Exchange, is required for public clients and increasingly recommended for confidential ones too, rather than being only a mobile and browser measure as it was originally introduced:

```
1. Client generates a random verifier, sends its hash (the challenge) with the
request
2. User authenticates at the authorization server and consents
3. Authorization server redirects back with a short-lived authorization code
4. Client exchanges the code, presenting the original verifier
5. Server checks the verifier hashes to the challenge, and returns the tokens
```

The code goes through the browser and the token exchange does not, so a code intercepted in the redirect is useless without the verifier, which never left the client.

Flow	For
Authorization Code + PKCE	Interactive login, all client types
Client Credentials	Machine to machine. No user involved
Device Authorization	Devices with no browser or keyboard, such as a television
~~Implicit~~	Deprecated. Returned tokens in the URL fragment
~~Resource Owner Password~~	Deprecated. The application handled the password directly

The two deprecated ones are worth knowing precisely because interviewers ask why they went away: Implicit exposed tokens in the URL where they reach history and logs, and Resource Owner Password required the user to give their password to the application, which is exactly what OAuth exists to avoid.

Scopes are not permissions. A scope says what the client application was granted on the user's behalf. It does not say the user is allowed to do it. Both checks are needed, and skipping the second is a route into section 15.

9. Authorization

Deciding what an authenticated identity may do, and the source of the most common serious vulnerabilities in real applications.

Three models, in increasing flexibility:

Model	How it decides
RBAC, Role-Based Access Control	Permissions attach to roles, roles attach to users. Simple, and coarse
ABAC, Attribute-Based Access Control	A rule over attributes of the subject, resource, action and context. Flexible, and harder to reason about
ACL, Access Control List	Permissions listed per object. Natural for per-document sharing

Many systems combine role-based permissions with ownership or relationship checks, and those can be modelled as ABAC, as **ReBAC**, Relationship-Based Access Control, or simply as resource-based policy, depending on how the system expresses them. The label matters less than the second half being present at all.

The distinction that matters more than the model:

> **Role checks are not enough. Most real failures are object-level.**

Confirming somebody has the `customer` role says nothing about whether *this* customer may read *that* invoice. Every request touching a specific object needs a check tying the identity to the object.

```
// Role check only. Any customer can read any invoice by changing the id.
@PreAuthorize("hasRole('CUSTOMER')")
public Invoice get(String invoiceId) {
    return invoices.findById(invoiceId).orElseThrow();
}

// Object-level check. Ownership is part of the decision.
@PreAuthorize("hasRole('CUSTOMER')")
public Invoice get(String invoiceId, Principal principal) {
    Invoice invoice = invoices.findById(invoiceId).orElseThrow();
    if (!invoice.customerId().equals(principal.getName())) {
        throw new AccessDeniedException("invoice belongs to another customer");
    }
    return invoice;
}
```

Better still, make the ownership part of the query rather than a check after the fact, so there is no window in which the wrong object has been loaded and no way to forget the check:

```
// The identity is in the query. A wrong id returns nothing rather than somebody
else's invoice.
Optional<Invoice> invoice = invoices.findByIdAndCustomerId(invoiceId,
principal.getName());
```

Two more habits: **deny by default**, so a new endpoint without an explicit rule is closed rather than open; and **enforce in one place**, because authorization scattered across controllers is authorization that will be inconsistent.

10. The OWASP Top 10

A periodically revised list of the most significant web application security risks, compiled by the Open Worldwide Application Security Project. Worth knowing as a **shared vocabulary** rather than as a checklist, and worth being clear in an interview that it is a starting point rather than a definition of secure.

The 2025 edition is the current one, and it differs enough from the 2021 list that quoting the older one is noticeable:

	Category	In one line
A01	Broken Access Control	Acting on objects or functions we are not entitled to. Section 16
A02	Security Misconfiguration	Defaults left in place, unnecessary features enabled, verbose errors
A03	Software Supply Chain Failures	The dependencies, tooling and pipelines we build on. Section 23
A04	Cryptographic Failures	Sensitive data unprotected in transit or at rest, or protected badly

	Category	In one line
A05	Injection	Untrusted input interpreted as code or query. Section 11, includes XSS
A06	Insecure Design	The flaw is in the design, so no amount of correct implementation fixes it
A07	Authentication Failures	Weak credentials, broken session handling, poor recovery
A08	Software or Data Integrity Failures	Unverified updates, unsafe deserialization, tampered artefacts
A09	Security Logging and Alerting Failures	The breach is real and nobody notices. Section 23
A10	Mishandling of Exceptional Conditions	Error and failure paths behaving unsafely

What moved, and why it is worth being able to say:

- **Broken Access Control stayed first.** It is the most frequently found category, and it is a design and discipline problem rather than one a library solves.
- **Security Misconfiguration rose to second**, from fifth, which reflects how much of a modern system is configuration rather than code.
- **Software Supply Chain Failures replaced and widened "Vulnerable and Outdated Components".** The category is no longer only about a dependency with a published advisory; it covers the build system, the tooling and the distribution path, which is where several significant compromises have actually occurred.
- **Mishandling of Exceptional Conditions is new**, and it is the list formalising the failure this document's section 3 describes: error paths, retries and fallbacks that behave unsafely, including permission checks that fail open.
- **SSRF is no longer its own entry.** It was A10 in 2021 and is now folded into the broader categories. It remains a serious and commonly exploited vulnerability, which is why section 16 still covers it in full, and the list changing is a statement about category structure rather than about the risk going away.
- **Logging gained "Alerting"**, which sharpens the point that collecting the events is not the control. Somebody has to be told.

11. Injection

Injection happens when untrusted input is interpreted as code rather than data. The family covers SQL, operating system commands, LDAP, XPath, expression languages and templates, and the mechanism is identical each time: a string is assembled and then handed to something that parses it.

```
// Vulnerable. The input becomes part of the statement.
String sql = "SELECT * FROM users WHERE email = '" + email + "'";

// email = anything' OR '1'='1
//   -> SELECT * FROM users WHERE email = 'anything' OR '1'='1'
```

The fix is parameterized queries, and the reason they work is worth stating precisely, because it is what separates a real answer from a memorised one:

> The query and the data travel to the database **separately**. The database parses the statement first, with placeholders, and the values are supplied afterwards as data that can never be re-parsed as syntax.

```
// Safe. The value cannot become syntax, whatever it contains.
String sql = "SELECT * FROM users WHERE email = ?";
try (PreparedStatement ps = connection.prepareStatement(sql)) {
    ps.setString(1, email);
    ...
}
```

```
// JPA. Bound parameter, not concatenation.
@Query("SELECT u FROM User u WHERE u.email = :email")
Optional<User> findByEmail(@Param("email") String email);
```

Three things worth knowing beyond the basic answer:

- **Escaping is the weaker approach.** It depends on getting every context, character set and edge case right, and it has failed repeatedly in practice. Parameterization removes the class of problem rather than filtering it.
- **Parameters cannot bind identifiers.** Table names, column names and sort directions are part of the syntax, so `ORDER BY ?` will not work. Where those must vary, map the input against an **allowlist** of permitted values and use the mapped result, never the raw input.
- **Least privilege limits the damage.** An application account without schema rights turns a successful injection into a smaller problem than one that can drop tables.

The same shape applies to command execution. Passing arguments as an array to the process, rather than building a shell string, means the shell never parses them:

```
// Safe: no shell involved, so shell metacharacters in filename are inert.
new ProcessBuilder("/usr/bin/convert", filename, "out.png").start();
```

Path traversal is the same idea applied to the file system: input becomes part of a path rather than part of a query.

```
// Vulnerable. name = ../../../../etc/passwd escapes the intended directory.
Path file = Path.of("/var/app/uploads", name);
return Files.readAllBytes(file);
```

The reliable fix is to **resolve the path and then check that the result is still inside the intended directory**, rather than filtering for `..`, which fails against encoded, doubled and platform-specific variants:

```
Path base = Path.of("/var/app/uploads").toRealPath();
Path file = base.resolve(name).normalize().toRealPath();

if (!file.startsWith(base)) {                // resolved, so symlinks are followed
too
    throw new AccessDeniedException("outside the upload directory");
}
```

`toRealPath` matters because a symbolic link inside the directory can point outside it, and a check on the unresolved path would not notice. Stronger still where the design allows: **do not take a path from the user at all.** Store an identifier, look up the real location, and never let input reach the file system as a path.

12. Cross-site scripting

XSS is injection into a page, where attacker-supplied content is executed as script in another user's browser, inside that user's session. It sits under Injection in the current OWASP list for exactly that reason.

Type	How it arrives
Stored	The payload is saved server-side, in a comment or a profile field, and served to everyone who views it. Often the highest impact, because it persists and reaches many users without any of them clicking anything
Reflected	The payload is in the request and echoed back in the response, so it needs a crafted link
DOM-based	Never reaches the server. Client-side code reads something attacker-influenced and writes it into the page

What it enables is worth stating plainly, because "run some JavaScript" understates it: the script runs with the victim's session, so it can read the page, make authenticated requests as them, capture keystrokes, and rewrite what they see.

The fix is contextual output encoding. Encode data when it is written into the page, according to where it lands, because the escaping rules differ per context:

Context	Needs
HTML body	HTML entity encoding

Context	Needs
HTML attribute	Attribute encoding, and the attribute quoted
JavaScript	JavaScript string encoding. Interpolating into script is best avoided entirely
URL parameter	URL encoding
CSS	CSS encoding

Encoding for the wrong context is a common way to be vulnerable while appearing to have done the work.

Practical points:

- **Modern frameworks encode by default.** Thymeleaf, React and the rest escape interpolated values, and the vulnerabilities appear where that is bypassed: `th:utext`, `dangerouslySetInnerHTML`, `innerHTML`, and building HTML by string concatenation.
- **Where users must supply HTML**, sanitise with a maintained library against an allowlist of tags and attributes, rather than filtering for what looks dangerous.
- **Content Security Policy** is defence in depth, restricting where scripts may load from and whether inline script runs. It reduces the impact of a mistake rather than removing the need to encode.
- `HttpOnly` **cookies** limit what a successful XSS can take. They do not prevent it, and the script can still make requests as the user.

13. Cross-site request forgery

CSRF makes a victim's browser send an authenticated request the victim did not intend. It relies on one browser behaviour: cookies for a site are attached to requests to that site regardless of which page caused the request.

```
<!-- On the attacker's page. The victim's browser attaches their bank cookie. -->
<form action="https://bank.example/transfer" method="POST" id="f">
  <input type="hidden" name="to" value="attacker">
  <input type="hidden" name="amount" value="5000">
</form>
<script>document.getElementById('f').submit();</script>
```

The attacker never reads the response, and does not need to. The effect is the point.

The distinction from XSS is a fair follow-up question: **XSS runs the attacker's script on our site with the user's session; CSRF sends a request from a different site and gets the browser to authenticate it.** XSS defeats every CSRF defence, since script running on our origin can read the token.

Three defences, generally combined:

`SameSite` **cookies**, the browser-level defence:

Value	Behaviour
`Strict`	Never sent on cross-site requests, including ordinary navigation from another site
`Lax`	Sent on top-level navigations that are safe methods, not on cross-site POST. The modern browser default
`None`	Always sent, and requires `Secure`. Needed for genuine cross-site use

`Lax` by default removes most of the classic attack, and it is not complete cover: it does not separate subdomains of the same site, and it depends on the browser honouring it. So it is one layer rather than the answer, normally combined with the token below and with an origin check. Setting it explicitly rather than relying on the default is worth doing.

Synchronizer tokens, the traditional fix: the server issues a random token tied to the session, the form submits it, and the server checks it. The attacker's page cannot read it, because the same-origin policy prevents reading our page. Spring Security implements this and enables it by default for browser-facing applications.

Checking `Origin` and `Referer` on state-changing requests, as a supporting check.

Two scoping points: **CSRF applies to credentials the browser attaches automatically**, so cookies and HTTP authentication. An API where the client attaches a bearer token from JavaScript is largely not exposed, because the attacker's page cannot make our JavaScript add that header. And **safe methods should be safe**: if `GET` changes state, it is reachable by an image tag, and no token defence applies.

14. Browser security controls

The browser enforces several controls of its own, and a common interview failure is describing them as though they protected the server. They protect **the user**, and the distinction decides what still has to be done server-side.

The same-origin policy is the foundation. An origin is scheme, host and port together, and by default a page may not read a response from a different origin. It is why a CSRF attacker can cause a request and cannot read the reply, per section 13.

CORS, Cross-Origin Resource Sharing, is the mechanism for relaxing that:

```
Access-Control-Allow-Origin: https://app.example.com
Access-Control-Allow-Credentials: true
Access-Control-Allow-Methods: GET, POST
Access-Control-Allow-Headers: Content-Type, Authorization
```

> **CORS is not an access control**
>
> **CORS tells the browser which origins may read a response. It does not decide who may call the API.** For a **simple** cross-origin request the browser sends it, the server processes it, and the browser then withholds the response from the calling page. For a **preflighted** request the browser checks the policy first, and a failed preflight means the real request is generally never sent.
>
> So CORS protects a user's data from a hostile page in that user's browser. It does nothing about `curl`, a script, a mobile application, or any other non-browser client, because there is no browser to enforce it. **Authentication and authorization on the server remain the control.**

Two details worth having ready:

- **The preflight.** For requests that are not simple, the browser first sends `OPTIONS` asking whether the real request is permitted. A common bug is a server that answers preflights correctly and does not apply the same authorization to the real request.
- **`Allow-Origin: *` cannot be combined with credentials.** Reflecting whatever `Origin` arrives, in order to work around that, effectively allows every origin, and with `Allow-Credentials: true` it hands authenticated responses to any site the user visits.

Security response headers, each closing a specific attack:

Header	What it does
`Content-Security-Policy`	Restricts where scripts, styles and frames may load from, and whether inline script runs. The main structural defence against XSS impact
`Strict-Transport-Security`	Tells the browser to use HTTPS for this host from now on
`X-Content-Type-Options: nosniff`	Stops the browser guessing a content type, so an uploaded file is not reinterpreted as script
`Referrer-Policy`	Limits how much of the current URL is sent onward, keeping identifiers and tokens out of other sites' logs
`Permissions-Policy`	Controls access to camera, microphone, geolocation and similar
`frame-ancestors` in CSP	Controls who may frame the page. The current defence against clickjacking, replacing `X-Frame-Options`

Clickjacking is the attack `frame-ancestors` addresses: our page is loaded invisibly inside the attacker's, positioned so the user's click lands on our control while they believe they are clicking something else. The user genuinely performed the action, so no CSRF token helps, which is why the fix is refusing to be framed.

15. Broken access control

First on the OWASP list, and the category most likely to be found in a real application, because it cannot be solved by a library. Each check is a decision somebody has to remember to make.

The shapes it takes:

Failure	Example
Insecure direct object reference	`/api/invoices/1043` returns the invoice regardless of who asks
Missing function-level check	An administrative endpoint that is simply not linked in the interface for ordinary users
Privilege escalation via parameter	A profile update accepting a `role` field from the request body
Path or method gaps	The check is on `POST /orders` and not on `PUT /orders/{id}`
Client-side enforcement	The button is hidden. The endpoint is not protected

The pattern behind most of them is **checking the route rather than the object**, which section 9 covers, or checking somewhere the attacker does not have to go through.

```
// Mass assignment: the request body binds straight onto the entity.
@PutMapping("/profile")
public User update(@RequestBody User user) {        // includes role, enabled, id
    return users.save(user);
}

// Bind to a request object holding only what a user may change.
@PutMapping("/profile")
public UserResponse update(@RequestBody UpdateProfileRequest request, Principal
principal) {
    return users.updateProfile(principal.getName(), request.displayName(),
request.locale());
}
```

Three practices that reduce this class:

- **Deny by default**, so an endpoint with no explicit rule is closed. A missing annotation then fails visibly rather than silently opening a route.
- **Scope queries by identity**, per section 9, so the wrong object is never loaded at all.
- **Test authorization deliberately.** The testing note makes the point that covering only the unauthenticated case leaves 403 untested, and the authenticated-but-not-permitted case is exactly the one that matters here.

Do not rely on identifiers being unguessable, and be precise about why: an unguessable identifier is a useful additional layer, and it is not an access control. Identifiers appear in logs, in referrer headers, in shared links and in browser history. Random identifiers plus a real check is the combination worth having.

16. Server-side request forgery

SSRF makes our server fetch a URL the attacker chooses. It matters far more than it appears, because the server sits inside the network and can reach places the attacker cannot.

```
// Vulnerable: the destination comes from the request.
@PostMapping("/import")
public String importFrom(@RequestParam String url) {
    return restClient.get().uri(url).retrieve().body(String.class);
}
```

Any feature taking a URL is a candidate: importing from a link, a webhook destination, fetching a profile image, rendering a preview, converting a document.

What it reaches:

- **Internal services** with no external exposure and often no authentication, on the assumption that being unreachable was enough.
- **Cloud instance metadata**, at `169.254.169.254`, which in older configurations returns credentials for the instance's role. This is the classic escalation from a page preview to cloud access.
- **The local machine**, through `localhost` and `file://`, reaching admin interfaces bound to the loopback address.

Defending it takes several layers, because each individually is bypassable:

- **Allowlist destinations** where the feature permits it. Where the set of legitimate destinations is knowable, this is the strongest primary control, and it is possible more often than assumed: webhooks usually target a customer domain that could be registered in advance.
- **Block internal ranges**, including loopback, link-local (`169.254.0.0/16`), and private ranges. Resolve the hostname and check the **resolved address**, not the string, since a hostname can point anywhere.
- **Handle redirects**, by validating every hop rather than only the first. A permitted URL that redirects to the metadata endpoint defeats a check done once.
- **Disable unneeded protocols**, so `file://`, `gopher://` and `ftp://` are not available through the fetcher.
- **Require authentication between internal services**, which is the defence in depth that holds when the rest fails, and the reason a zero-trust posture matters.
- **Use IMDSv2** or the equivalent, which requires a session token obtained with a `PUT` and so is not reachable by a simple attacker-triggered `GET`.

A caution worth stating: **the resolve-then-check approach has a race**, because the name can resolve differently between the check and the request, which is DNS rebinding. Doing the check properly means resolving once and connecting to that address, or using a proxy that enforces the policy.

17. File uploads

A frequent interview scenario, because an upload endpoint concentrates several problems at once: attacker-controlled content, attacker-controlled metadata, and storage that something later reads.

What is attacker-controlled, and often trusted anyway:

Field	Why it cannot be trusted
Filename	May contain path segments, null bytes, or a second extension
`Content-Type` header	Set by the client. It says whatever the client wants
Extension	Bears no relationship to the contents
Size declared in headers	Only the bytes actually received are real

The controls, roughly in order of how much they matter:

- **Do not use the supplied filename.** Generate one, store the original as a display label only, and never let it reach the file system as a path. This removes path traversal, per section 11, and the encoding tricks around it.
- **Validate the content, not the label.** Check the actual bytes against the expected format rather than trusting the extension or `Content-Type`. A file named `avatar.png` can be anything.
- **Limit size**, and enforce it as the bytes arrive rather than after reading the whole thing into memory, since the latter is a denial-of-service route on its own.
- **Store outside the web root**, and outside anywhere the application would execute. An uploaded file reachable as a script is remote code execution.
- **Serve it back safely.** Set an explicit `Content-Type`, send `X-Content-Type-Options: nosniff` so it is not reinterpreted, and use `Content-Disposition: attachment` for anything not deliberately rendered. Serving user content from a separate origin limits what a malicious file can do with our session.
- **Authorize the download**, not only the upload. An upload endpoint with a careful check and a download endpoint that serves any identifier is the access-control failure from section 15 in a different place.
- **Scan where the risk warrants it**, for anything that will be redistributed to other users.

The point that connects them: an upload is not one decision but three, taken at different times. **What we accept, where we put it, and how we give it back**, and a control at only one of the three leaves the other two open.

18. Encoding, hashing and encryption

Three words used interchangeably in conversation and meaning entirely different things, which makes this a quick way for an interviewer to calibrate.

	Purpose	Can it be undone	Key
Encoding	Represent data in another format for transport or compatibility	Yes, by anyone	No
Hashing	Produce a fixed-size fingerprint	Not by design	No, though HMAC adds one
Encryption	Keep data confidential from those without the key	Yes, by a party holding the key	Yes

> **Encoding is not security.** Base64, URL encoding and hex are reversible by anybody, with no secret involved. Treating base64 as though it conceals something is a recognisable mistake, and it is why the JWT point in section 7 matters.

Hashing is designed to be computationally infeasible to reverse. That is a different claim from "impossible", and the difference is the whole of section 5: an attacker cannot invert the function, and can absolutely guess inputs and compare the resulting hashes, which is why a fast general-purpose hash over passwords is dangerous and a deliberately expensive one is not. Its uses are integrity checking, deduplication, and password storage with the appropriate family. A good hash is collision-resistant, which is why MD5 and SHA-1 are no longer acceptable for security purposes.

Encryption splits two ways:

	Core idea	Typical uses
Symmetric	One shared key both encrypts and decrypts. AES	Bulk data. Fast
Asymmetric	A public and private key pair. RSA, elliptic curve	Key agreement, signatures, identity

Resist the shorthand that asymmetric means "the public key encrypts and the private key decrypts". RSA can work that way, and the elliptic-curve algorithms in common use do not encrypt at all: ECDH performs key agreement and ECDSA and EdDSA produce signatures. **A key pair is the idea; encryption is only one thing that can be built from it.**

Asymmetric operations are far more expensive, so protocols use them for the parts that need a key pair and switch to symmetric for the data. **TLS uses asymmetric cryptography to authenticate the parties and to agree a key, then protects the application data with symmetric authenticated encryption.** In TLS 1.3 that agreement is an ephemeral Diffie-Hellman exchange rather than one side encrypting a key to the other, and the certificate's key signs the handshake rather than transporting a secret.

Two related ideas that complete the set:

- **A digital signature uses an asymmetric key pair, with the private key signing and the public key verifying**, giving integrity and origin rather than confidentiality. This is the `RS256` case in section 7.
- **An HMAC** is a keyed hash, giving integrity and origin using a shared secret. Cheaper than a signature, and both parties can produce it, so it does not prove which one did.

Use **authenticated encryption** such as AES-GCM, rather than encryption alone. Encryption without integrity permits an attacker to alter ciphertext in ways that produce meaningful plaintext changes, and combining the two by hand is exactly the kind of construction section 3 warns against.

19. Transport security

TLS provides three things, and being able to list them separately is worth marks because people usually name only the first:

Property	Meaning
Confidentiality	Nobody in the path can read the traffic
Integrity	Nobody in the path can alter it undetected
Server authentication	The client verifies it is talking to the intended server, via a certificate chain to a trusted authority

The third is what makes the first two worth anything, and it is the one applications break. **Disabling certificate validation to make a test environment work** removes the authentication, which reduces TLS to encryption with an unknown party. That change reaching production is a common route to a machine-in-the-middle position.

Mutual TLS adds client authentication, with both sides presenting certificates. Common between internal services, where it also gives each service a verifiable identity.

Related controls worth naming:

- **HSTS**, HTTP Strict Transport Security: a response header instructing the browser to use HTTPS for this host from now on, which closes the window where an initial plain-text request could be intercepted and redirected.
- **Certificate lifetimes and rotation.** Expiry causes outages more often than compromise does, so automated renewal is an availability control as well as a security one.
- **TLS termination.** Terminating at a load balancer leaves traffic behind it unencrypted unless re-encrypted, and whether that is acceptable is a decision about the trust boundary, per section 2, rather than a default.

Encryption in transit and encryption at rest answer different threats, and conflating them is common. In transit protects against somebody on the network path. At rest protects against somebody obtaining the storage: a stolen disk, a mislaid backup, a snapshot copied out of an account. Neither protects against an

application with valid credentials reading data it should not, which is section 15's problem.

20. Dependencies and supply chain

Most of the code in a modern application was written by somebody else, and it runs with the application's full privileges. Log4Shell demonstrated the scale: a logging library, present transitively in an enormous number of applications, taking remote code execution from an attacker-supplied string.

The specific risks:

Risk	Description
Known vulnerabilities	A dependency with a published advisory, still in use
Transitive depth	The vulnerable package is a dependency of a dependency, and nobody chose it
Typosquatting	A package named close to a popular one, published to be installed by mistake
Compromised package	A legitimate package whose maintainer account or pipeline was taken over
Compromised pipeline	The build system itself, which produces artefacts everyone trusts

The practices that address them:

- **Scan continuously**, not once. A dependency safe at the time it was added becomes vulnerable when an advisory is published, so the check has to run against the current advisory database on a schedule as well as in the build.
- **Lock versions**, so a build is reproducible and an unexpected version cannot appear between builds.
- **Keep an inventory.** An SBOM, Software Bill of Materials, is the list of what is actually in a build, and it is what makes the question "are we affected by this advisory" answerable in minutes rather than days. That is the concrete lesson from Log4Shell, where the expensive part for most organisations was not patching but finding out where the library was.
- **Update routinely.** Small regular upgrades are less risky than a large one under time pressure during an incident.
- **Reduce the count.** A dependency added for one function is a permanent obligation, and the cheapest vulnerability to manage is the one not introduced.

Not every advisory is urgent, and saying so is a sign of judgement rather than laziness. Severity scores describe the vulnerability in general, not our exposure to it: a critical rating in a code path we never call is less pressing than a moderate one in the request handler. What matters is reachability and exposure, and having a way to make that judgement quickly is more valuable than treating every advisory as an emergency.

21. Secrets in an application

Covered from other angles in the Git and cloud notes, so this is the application's own view of it.

A secret's value should never be in the artefact. Not in source, not in a configuration file that ships with the build, not baked into a container image, and not written into a pipeline definition. The reason is that all of those are copied, cached and shared far more widely than anyone tracks.

The distinction is between the value and a reference to it. A pipeline definition legitimately *names* a secret, and the value is injected at run time by whatever holds it:

```
env:
  DATABASE_PASSWORD: ${{ secrets.DATABASE_PASSWORD }}   # a reference, not the value
```

Where they belong instead is a secret manager, with the application fetching at start or on demand, and with access controlled per service identity rather than by a shared credential.

```
// Wrong: in the artefact, and in version control.
private static final String API_KEY = "sk_live_51H...";

// Better: injected from the environment, populated by a secret manager.
@Value("${payment.api-key}")
private String apiKey;
```

The environment variable is an improvement rather than an end state. Depending on the platform and the application, environment contents can be exposed through process inspection, crash diagnostics, or a debugging endpoint that dumps configuration, so a managed secret fetched over an authenticated channel is stronger.

Four practices worth stating:

- **Rotate on a schedule**, and make rotation routine rather than an emergency procedure. A rotation path exercised quarterly works when it is needed urgently.
- **Assume exposure is permanent.** Per the Git note, removing a pushed secret from history does not un-expose it. Rotation is the fix and cleanup is tidying.
- **Scan the pipeline**, so a commit containing a credential is refused rather than reviewed.
- **Keep them out of logs.** A request logger that dumps headers writes the authorization token to a log with much wider access than the secret store, which is a common and quiet leak. Section 23 has the general form of this.

22. Rate limiting and abuse prevention

Not every attack exploits a defect. Some just use a working feature far more than it was designed for, and the absence of a limit is the vulnerability. The current OWASP list treats this under insecure design, since no line of code is wrong.

Where limits matter, and what the attack looks like:

Target	The abuse
Login	Brute force against one account, or credential stuffing across many
One-time codes	Guessing a six-digit code, which is trivial without a limit on attempts
Password reset	Flooding a mailbox, or probing which addresses are registered
Registration	Bulk account creation
Expensive endpoints	Search, export, report generation. Resource exhaustion at low request volume
Any read endpoint	Scraping the dataset one record at a time

What to limit by is the design decision, and getting it wrong is how limits become useless or harmful:

- **By account** for authenticated actions, which is precise.
- **By address** for unauthenticated ones, remembering that shared addresses mean legitimate users behind an office or a mobile network share a bucket.
- **By target rather than by source** for login and reset, since limiting per source lets a distributed attempt through, while limiting attempts against a single account catches it. Both together is normal.

Two behaviours worth naming:

- **Prefer slowing down to locking out.** An increasing delay costs an attacker almost everything and a legitimate user very little, whereas a hard lockout on a named account is itself a denial-of-service route against that account.
- **Decide the limiter's failure mode deliberately.** If the component tracking limits is unreachable and the application simply continues, the protection disappears exactly when it is most likely to be needed. Failing closed everywhere is not the answer either, because a limiter outage then becomes a full outage. The workable split is by what the operation protects: **fail closed, or fall back to a restrictive local limit, for login, one-time codes, password reset and anything expensive to create**; for ordinary reads, degrade to a coarse local limit and alert, rather than either rejecting everything or silently removing the control.

One boundary worth stating: **rate limiting controls abuse of an operation somebody is entitled to perform. It is not a substitute for authorization**, in the same way CORS is not, and a limited endpoint that anyone may call is still an endpoint anyone may call.

The general form is in the API and distributed systems notes, which cover the algorithms and the response headers. The security-specific part is knowing **which operations need a limit because of what they protect** rather than because of what they cost.

23. Security logging and monitoring

Ninth on the OWASP list, and there because **the failure is not being attacked, it is not knowing**. Long detection times are what turn an intrusion into a breach, since the attacker has time to work.

What is worth logging, given that the point is reconstructing what happened:

Event	Why
Authentication attempts, success and failure	Credential stuffing appears here first
Authorization denials	A pattern of them is somebody probing
Changes to permissions, roles and account state	The step an attacker takes after gaining access
Access to sensitive data	Who read what, for the questions asked afterwards
Administrative actions	The highest-value events
Input validation failures at scale	Somebody testing what the system rejects

Each needs enough context to be useful: who, what, when, from where, and whether it succeeded. A log saying "access denied" without the identity or the resource cannot answer anything later.

What must not be logged matters equally, because logs are widely readable, retained for a long time, and shipped to third-party systems:

- Passwords, including in failed-login handlers that log the attempt with parameters.
- Session identifiers, tokens and API keys, which turn a log reader into an authenticated user.
- Full payment card numbers, government identifiers, and personal data beyond what is needed.
- Whole request bodies and header dumps on error paths, which is how the items above usually arrive by accident.

Two more properties:

- **Logs need protecting.** An attacker with write access to logs can erase their traces, so shipping to a separate system with tamper-resistant or immutable retention is what makes them trustworthy as evidence rather than a narrative the attacker controls.
- **Alerting is what makes logging a control.** A log nobody reads detects nothing. The general form is in the observability note; the security-specific part is that the signals worth alerting on are the ones above, and that they should reach somebody who can act rather than a dashboard.

24. Interview questions

As in the other notes, each question has an **opening** of roughly thirty to forty seconds, whose last sentence points at the deeper material, and a **follow-up** for when the interviewer takes it.

"What is the difference between authentication and authorization?"

> Authentication is establishing who somebody is; authorization is deciding what they may do, and it happens after. Multi-factor authentication belongs to the first and only counts as multi-factor when the factors come from different categories, so something known, something held, something inherent. Two passwords are one factor twice. The reason the distinction matters practically is that authorization is a major source of serious application vulnerabilities, precisely because authenticating somebody says nothing about which objects or actions they may reach.

If they go deeper:

> Broken access control is first on the OWASP list, and the reason is that no library solves it: every check is a decision somebody has to remember to make. The specific failure is usually checking the role rather than the object. Confirming somebody has the customer role says nothing about whether this customer may read that invoice, so an endpoint like slash invoices slash 1043 hands over any invoice to anybody who changes the number. My preferred fix is to put the identity into the query rather than checking afterwards, so a wrong identifier returns nothing rather than returning somebody else's data, and there is no check to forget.

"How would you store passwords?"

> Hashed, never encrypted, because encryption implies a key that can reverse it and there is no legitimate reason to recover the original. And with a hash designed for passwords, so Argon2id as the current general recommendation, or bcrypt or scrypt. Not SHA-256, and the reason is exactly what makes SHA-256 good elsewhere: it is fast, and hardware computes billions per second, so a leaked table of those hashes becomes a list of passwords shortly afterwards.

If they go deeper:

> Password hashes are deliberately expensive, with a work factor that gets raised as hardware improves, which is why the parameters need revisiting rather than being set once. They differ in how the cost is imposed, and I would be precise about that: Argon2id and scrypt are memory-hard, requiring configurable memory as well as processor time, which is what makes cracking on specialised hardware costly; bcrypt is deliberately slow without being memory-hard in the same sense; and PBKDF2 raises cost through iteration count alone, which makes it the weakest of the four against modern hardware. The salt is a unique random value per password, stored alongside the hash and not secret, and its job is to make identical passwords hash differently so precomputed tables do not work and one cracked hash does not reveal every account sharing that password. Modern libraries generate and embed it for us. A pepper is a separate secret held outside the database, so a database leak alone is not enough to start cracking, and it is optional and complicates rotation. The other detail is to verify through the library's own method rather than fetching the hash and comparing it ourselves, since established implementations handle the comparison semantics including the timing differences an early-exit check would leak.

"What is SQL injection and how do you prevent it?"

> It is untrusted input being interpreted as query syntax rather than as data, which happens whenever a query is assembled by string concatenation. Passing something like `anything' OR '1'='1` into a concatenated `WHERE` clause changes what the statement means. The fix is parameterized queries, and the reason they work is the part worth being precise about: the query and the values travel to the database separately.

If they go deeper:

> The database parses the statement first with placeholders in it, then the values arrive as data that can never be re-parsed as syntax, so it does not matter what they contain. That is why parameterization removes the class of problem while escaping only filters it, and escaping has failed repeatedly because it depends on getting every context and character set right. Two limits worth knowing: parameters cannot bind identifiers, so a dynamic `ORDER BY` column needs an allowlist mapping rather than the raw input; and least privilege still matters, because an application account without schema rights makes a successful injection a much smaller event. The same shape applies to command execution, where passing arguments as an array means no shell parses them.

"Explain XSS and CSRF, and how they differ."

> XSS is getting our own site to run attacker-supplied script in another user's browser, so it executes with that user's session and can read the page, make authenticated requests and capture input. CSRF is different: the attacker's own site causes the victim's browser to send an authenticated request to ours, relying on the browser attaching cookies regardless of which page triggered the request. So XSS runs code on our origin, and CSRF sends a request from elsewhere and gets the browser to authenticate it.

If they go deeper:

> The fix for XSS is contextual output encoding, encoding data according to where it lands in the page, since HTML body, attribute, JavaScript and URL contexts all have different rules, and encoding for the wrong one is a common way to look protected while not being. Frameworks encode by default, so the vulnerabilities cluster around the escape hatches like `innerHTML` or `dangerouslySetInnerHTML`. Content Security Policy reduces the impact and does not replace encoding. For CSRF the browser-level defence is `SameSite` cookies, `Lax` being the modern default, and I would treat that as one layer rather than the answer, since it does not separate subdomains and depends on the browser honouring it. Alongside it, synchronizer tokens, which the attacker's page cannot read because of the same-origin policy, and an origin check. The connection worth stating is that XSS defeats every CSRF defence, since script on our origin can read the token, so they are not independent.

"What is a JWT, and what are the pitfalls?"

> Three base64url-encoded parts, header, payload and signature. The signature proves the token was issued by the holder of the signing key and has not been altered, which lets a server verify it without storing anything, and that statelessness is the whole appeal. The thing people get wrong is assuming it is confidential: base64url is encoding, not encryption, so anybody holding the token can read every claim with no key involved. So nothing confidential belongs in the payload, though the identity claims it exists to carry, `sub`, `iss`, `aud` and `exp`, are not secrets and belong there.

If they go deeper:

> The cost of statelessness is revocation. A token valid for an hour stays valid for an hour after the account is disabled, unless we check a revocation list, which reintroduces exactly the lookup the design avoided. The usual arrangement accepts that: a short-lived access token measured in minutes, plus a long-lived refresh token stored server-side that can be revoked. The specific failures worth naming are `alg: none`, where the token declares itself unsigned and a naive library accepts it; algorithm confusion, where a token signed HS256 is verified by a server expecting RS256 using the public key as the HMAC secret, so anybody can forge one; decoding without verifying, which are two similar-looking library calls; and not checking `exp`, `iss` and `aud`. Storing it in `localStorage` also turns any XSS into token theft, where an `HttpOnly` cookie would not be readable.

"Is OAuth 2.0 an authentication protocol?"

> No, and that is the point of the question. OAuth 2.0 is an authorization framework: it answers whether an application may act on a user's behalf for a given scope, and deliberately says nothing about who the user is. OpenID Connect is the identity layer built on top, adding an ID token stating who authenticated, when and how. So "Sign in with ..." is OIDC, and using OAuth alone for login is a known anti-pattern, because an access token proves access rather than identity.

If they go deeper:

> The flow to know is Authorization Code with PKCE, which is the recommended interactive authorization flow. PKCE is required for public clients and increasingly recommended for confidential ones too, rather than being only the mobile and browser measure it was introduced as. The client generates a random verifier, sends its hash up front, and presents the original verifier when exchanging the code, so a code intercepted in the redirect is useless without a value that never left the client. Client Credentials is the machine-to-machine flow with no user involved. Implicit and Resource Owner Password are both deprecated, and the reasons are instructive: Implicit returned tokens in the URL fragment where they reach history and logs, and Resource Owner Password required handing the user's password to the application, which is precisely what OAuth exists to avoid. One thing I would add is that scopes are not permissions: a scope says what the client was granted on the user's behalf, not that the user is entitled to it, so both checks are needed.

"Does CORS protect my API?"

> No, and that is worth being direct about because it is a common misunderstanding. CORS tells the **browser** which origins are allowed to read a response. For a simple cross-origin request the request still reaches the server and is processed, and the browser withholds the response from the calling page; for a preflighted one the browser checks first and may never send the real request. Either way it protects a user's data from a hostile page in that user's browser, and it is not enforced by `curl`, a script or a mobile client. Authentication and authorization on the server remain the actual control.

If they go deeper:

> The underlying thing CORS relaxes is the same-origin policy, which is also why a CSRF attacker can cause a request and not read the reply. Two details I would watch for: the preflight, where the browser sends `OPTIONS` first for non-simple requests, and a common bug is a server that answers preflights correctly but does not apply the same authorization to the real request. And `Access-Control-Allow-Origin: *` cannot be combined with credentials, so teams work around that by reflecting whatever `Origin` arrives, which effectively allows every origin. Combined with `Allow-Credentials: true`, that hands authenticated responses to any site the user happens to visit.

"What is SSRF and why does it matter?"

> It is getting our server to fetch a URL the attacker chooses, and it matters far more than it looks because the server sits inside the network and can reach things the attacker cannot. Any feature that takes a URL is a candidate: importing from a link, a webhook destination, a profile image, a link preview. The classic escalation is the cloud instance metadata endpoint at 169.254.169.254, which in older configurations returns credentials for the instance's role, so a preview feature becomes cloud access.

If they go deeper:

> Where the set of legitimate destinations is knowable, an allowlist is the strongest primary control, and it is more often possible than people assume, since a webhook usually targets a customer domain that can be registered in advance. Where it is not, we block internal ranges including loopback and link-local, and critically we check the **resolved** address rather than the hostname string, because a name can point anywhere. Redirects have to be validated at every hop, or a permitted URL that redirects to the metadata endpoint defeats a check done once. There is also a race in resolve-then-connect, which is DNS rebinding, so doing it properly means connecting to the address we checked or using a proxy that enforces the policy. The layer that holds when all of that fails is requiring authentication between internal services, which is the argument for not treating the network perimeter as a control.

"How would you secure a file upload endpoint?"

> I would treat it as three separate decisions rather than one, because a control at only one of them leaves the others open: what we accept, where we put it, and how we give it back. On accepting, everything the client sends about the file is attacker-controlled, so the filename, the extension and the `Content-Type` header all say whatever the client wants. I would generate the stored filename rather than use theirs, and validate the actual bytes against the expected format rather than trusting the label.

If they go deeper:

> Generating the name removes path traversal along with the encoding tricks around it, and if a path has to be built from anything user-supplied I would resolve the canonical path and verify it is inside the canonical base directory rather than filtering for dot-dot. Size gets enforced as bytes arrive rather than after reading the whole thing into memory, since the latter is a denial-of-service route by itself. Storage goes outside the web root and outside anywhere the application could execute, because an uploaded file reachable as a script is remote code execution. On the way back, an explicit `Content-Type` with `nosniff` so it is not reinterpreted, `Content-Disposition: attachment` for anything not deliberately rendered, and ideally a separate origin so a malicious file cannot act with our session. And the download needs authorizing, not only the upload, since a careful upload check with an open download endpoint is just object-level access control failing somewhere else.

"What happens if your rate limiter fails?"

> That is a design decision rather than an accident, and I would want it made deliberately. If the component tracking limits is unreachable and the application just carries on, the protection disappears at exactly the moment it is most likely to be needed. But failing closed everywhere is not the answer either, because then a limiter outage becomes a full outage, and we have converted an abuse problem into an availability one.

If they go deeper:

> The split I would make is by what the operation protects. For login, one-time code verification, password reset and anything expensive to create, I would fail closed or fall back to a restrictive local limit, because the cost of letting those through unlimited is much higher than the cost of rejecting some legitimate traffic. For ordinary read endpoints I would degrade to a coarse in-process limit and alert loudly, rather than either rejecting everything or silently removing the control. The general point is that a fallback which quietly disables a security control is the same failure as the permission check that catches an exception and returns true. I would also say that rate limiting controls abuse of an operation somebody is entitled to perform, so it never substitutes for authorization, in the same way CORS does not.

"Encoding, hashing, encryption. What is the difference?"

> Encoding changes the representation for transport or compatibility and is undone by anyone with no key, so base64 is not security. Hashing produces a fixed-size fingerprint and is designed to be computationally infeasible to reverse, which is not the same as impossible, since an attacker can still guess inputs and compare hashes. That is exactly why password storage needs a deliberately expensive hash. Encryption is designed to be undone by a party holding the key, and is the only one of the three that provides confidentiality. Calling base64 encryption is a mistake an interviewer will notice immediately, and it is why the JWT payload point matters.

If they go deeper:

> Encryption splits into symmetric, one shared key both ways as with AES, which is fast and suits bulk data; and asymmetric, a public and private key pair, which is much more expensive and suits key agreement, signatures and identity. I would avoid the shorthand that asymmetric means the public key encrypts and the private key decrypts, because RSA can work that way and the elliptic-curve algorithms in common use do not encrypt at all: ECDH agrees a key and ECDSA or EdDSA sign. The key pair is the idea, and encryption is only one thing built from it. TLS uses asymmetric cryptography to authenticate the parties and agree a key, then protects the application data symmetrically because that is far more efficient. I would be careful not to describe it as one side encrypting a key to the other, since in TLS 1.3 the agreement is an ephemeral Diffie-Hellman exchange and the certificate's key signs the handshake rather than transporting a secret. A digital signature uses an asymmetric key pair the other way round, with the private key signing and the public key verifying, giving integrity and origin rather than confidentiality. An HMAC does the same with a shared secret, which is cheaper but does not prove which party produced it since both can. And I would use authenticated encryption like AES-GCM rather than encryption alone, because encryption without integrity lets an attacker alter ciphertext in meaningful ways, and combining the two by hand is the sort of construction that goes wrong.

"How do you handle vulnerable dependencies?"

> By treating it as continuous rather than one-off, because a dependency that was safe when it was added becomes vulnerable the day an advisory is published, so scanning has to run on a schedule against the current advisory database and not only at the point of adding it. Alongside that, locked versions so builds are reproducible, and routine small upgrades rather than a large one under pressure during an incident. The piece people underrate is knowing what is actually in the build.

If they go deeper:

> That is what an SBOM is for, a software bill of materials, and it is what makes "are we affected by this advisory" answerable in minutes rather than days. Log4Shell was the demonstration: for most organisations the expensive part was not patching but finding out where the library was, since it was usually a transitive dependency nobody had chosen. I would also say that not every advisory is urgent, and that is judgement rather than laziness: a severity score describes the vulnerability in general and not our exposure to it, so a critical rating in a code path we never call is less pressing than a moderate one in the request handler. Having a fast way to make that call is worth more than treating every advisory as an emergency, because a team that treats everything as urgent stops responding to any of it.

"You are designing a new API endpoint. How do you think about security?"

> I would start at the trust boundaries rather than at a checklist, so what crosses into this endpoint, where it comes from, and what it can reach behind us. Then authentication, then authorization at the object level rather than the route level, then treating every input as attacker-controlled including headers and anything read back from the database that a user once wrote. Client-side validation I would treat as a usability feature and not a control, since the request can be made without the client.

If they go deeper:

> Concretely: parameterized queries for anything reaching the database, an allowlist for anything that has to vary the query structure, output encoded per context, deny by default so a missing rule closes the endpoint rather than opening it, and a bound request object rather than the entity so the body cannot set fields like `role`. Then the operational side: rate limiting, uniform error responses that do not reveal whether a record exists, no secrets or tokens in the logs, and logging the authorization denials because a pattern of them is somebody probing. Underneath all of it I would want defence in depth, so that no single control failing is sufficient, and least privilege on the credentials involved, so that a compromise is a limited one rather than a general one.

25. Common mistakes

- Trusting the client, so validation exists only in the browser and the endpoint accepts anything sent directly.
- Treating data as safe because of where it arrived from, rather than because it was checked. Headers, cookies, file names and values previously written by users are all input.
- Building queries by string concatenation, then adding escaping instead of parameterizing.
- Attempting to parameterize an identifier, such as a sort column, and falling back to concatenation when it does not bind, rather than using an allowlist.
- Hashing passwords with SHA-256, or with any general-purpose hash, when the speed that makes it good elsewhere is what makes it wrong here.
- Encrypting passwords instead of hashing them, which creates a key that turns a leak into plaintext.

- Reading a JWT payload and trusting it without verifying the signature, or letting the token's own `alg` header decide how it is verified.
- Putting anything sensitive in a JWT payload, on the assumption that base64url conceals it.
- Storing tokens in `localStorage`, so any XSS becomes account takeover.
- Using OAuth alone for login, where an access token proves access rather than identity.
- Treating a scope as a permission, when it says what the client was granted and not what the user is entitled to.
- Checking the role and not the object, so any authenticated user reads any record by changing an identifier.
- Binding a request body straight onto an entity, letting a user set `role` or `enabled`.
- Relying on an identifier being unguessable as though it were an access control.
- Hiding a button and leaving the endpoint unprotected.
- Encoding output for the wrong context, so the escaping looks done and the attribute or script context is still injectable.
- Letting `GET` change state, which puts it beyond every CSRF token defence.
- Disabling certificate validation to make something work in a test environment, and shipping it.
- Confusing encryption in transit with encryption at rest, and assuming either protects against an application reading data it should not.
- Fetching an attacker-supplied URL server-side with no allowlist, and checking the hostname string rather than the resolved address.
- Logging whole request bodies or header dumps on error paths, which writes tokens and personal data into a widely readable system.
- Scanning dependencies once at the point of adding them, rather than continuously against current advisories.
- Failing open, where an exception in a permission check is caught, logged and treated as permission granted.
- Treating CORS as an access control, when it tells the browser what it may read and does nothing about a non-browser client.
- Reflecting whatever `Origin` arrives in order to allow credentials, which permits every origin rather than the intended one.
- Answering a preflight correctly and then not applying the same authorization to the real request.
- Filtering for `..` to stop path traversal, rather than resolving the path and checking it is still inside the base directory.
- Trusting an upload's filename, extension or `Content-Type`, all of which the client chooses.
- Storing uploads somewhere the application can execute, or serving them without `nosniff`.
- Rate limiting the source address only, so a distributed attempt against one account is never noticed.
- Locking an account hard on failed attempts, which is a denial-of-service route against any named user.

- Quoting the 2021 OWASP Top 10 when the 2025 edition is current, which is an easy thing to be caught on.

26. Quick reference

Term	Meaning
Authentication against authorization	Who somebody is, against what they may do. In that order
CIA triad	Confidentiality, integrity, availability
STRIDE	Spoofing, tampering, repudiation, information disclosure, denial of service, elevation of privilege
Trust boundary	A line where the level of trust changes. Validate at each one
Defence in depth	Assume each control fails, so no single failure is sufficient
Least privilege	Exactly the access needed, and no more
Fail closed	When a check cannot complete, deny
Multi-factor	Factors from **different** categories: known, held, inherent
Salt	Unique per password, stored with the hash, not secret. Defeats precomputed tables
Pepper	A secret held outside the database, so a database leak alone is not enough
Argon2id, bcrypt, scrypt	Password hashes, deliberately expensive with a tunable work factor. **Argon2id and scrypt are memory-hard; bcrypt is not**
Session against self-contained token	Revocable and needs storage, against stateless and hard to revoke
`HttpOnly`	JavaScript cannot read the cookie, limiting what an XSS can take
`SameSite=Lax`	Withheld on cross-site subrequests and POSTs, allowed on top-level safe navigations such as GET. The modern browser default, and the reason GET must not change state
JWT	Header, payload, signature. Signed, **not encrypted**. Anyone can read the claims
`alg: none`	The token declaring itself unsigned. Pin the expected algorithm server-side
Algorithm confusion	HS256 token verified by an RS256 server using the public key as the secret
OAuth 2.0	An authorization framework. Not authentication
OpenID Connect	The identity layer on top, adding an ID token
Authorization Code + PKCE	The flow for interactive login, all client types
Client Credentials	Machine to machine, no user
RBAC, ABAC, ACL	Roles, attribute rules, per-object lists

Term	Meaning
IDOR	Reaching another user's object by changing an identifier
Parameterized query	Statement and values sent separately, so values can never become syntax
Contextual output encoding	Encode per destination: HTML body, attribute, JavaScript, URL, CSS
CSP	Restricts script sources. Reduces XSS impact, does not replace encoding
CSRF	The victim's browser sends an authenticated request they did not intend
Same-origin policy	Scheme, host and port together. A page may not read another origin's response
CORS	Tells the **browser** which origins may read a response. Enforced by browsers, so it does not protect an API from non-browser clients
Preflight	The `OPTIONS` request before a non-simple cross-origin call. Authorize the real request too
`nosniff`	Stops the browser guessing a content type, so an upload is not reinterpreted as script
`frame-ancestors`	Controls who may frame the page. The current clickjacking defence
Clickjacking	Our page framed invisibly so a real click lands on our control. No token defends it
Path traversal	Input becoming part of a path. Resolve, then check it is still inside the base directory
SSRF	The server fetches a destination the attacker chose
`169.254.169.254`	Cloud instance metadata. The classic SSRF escalation
Encoding, hashing, encryption	Representation anyone can undo, a fingerprint infeasible to reverse, and confidentiality undone by a key holder
Symmetric against asymmetric	One shared key and fast, against a key pair and expensive. Not "public encrypts, private decrypts"
HMAC against signature	Shared secret, against a private key that proves which party
Authenticated encryption	AES-GCM. Confidentiality and integrity together
mTLS	Both sides present certificates, giving each service a verifiable identity
HSTS	Tells the browser to use HTTPS for this host from now on
SBOM	The inventory of what is in a build. Turns advisory response into minutes
Rate limiting by target	Limit attempts against the account, not only the source, or a distributed attempt goes unnoticed
Limiter failure mode	Fail closed for login, codes and reset. Degrade to a coarse local limit for ordinary reads

Symptom or situation	What to reach for
A query built by concatenating input	`PreparedStatement` or a bound JPA parameter
A sort column or table name that must vary	Allowlist, and use the mapped value
User-supplied text rendered into a page	Encode for the context. Sanitise with a library if HTML is required
A cookie-authenticated state-changing form	`SameSite`, a synchronizer token, and never `GET`
An endpoint returning a record by identifier	Scope the query by the caller's identity
A profile update binding to the entity	Bind to a request object holding only editable fields
A feature that fetches a URL	Allowlist, resolve then check, validate every redirect hop
A file path built from user input	Resolve the canonical path, then verify it is within the canonical base directory
An upload endpoint	Generated filename, content validated not the label, stored outside the web root, `nosniff` on the way back
A login or one-time-code endpoint	Limit by target as well as by source, and slow down rather than lock out
"CORS is blocking us" in a ticket	Fix the policy deliberately. Never reflect arbitrary `Origin` with credentials allowed
A password to store	Argon2id, bcrypt or scrypt, via the framework's encoder
A token stored in the browser	`HttpOnly` cookie rather than `localStorage`
A permission check that might throw	Catch and **deny**, never catch and continue
An advisory against a dependency	Check reachability and exposure before treating the score as the priority
A secret that reached the remote	Rotate first. Cleanup is tidying, not the fix
An error response revealing whether an account exists	Uniform message and uniform timing for both cases